A Practical Guide to Maze-Solving Robot Navigation: Design Principles and Implementation Strategies

Tomás S. Giomi
Department of Robotics and Control
Instituo La Salle Florida
Email: tomasgiomi@gmail.com, tgiomi@lasalleflorida.edu.ar

***Abstract*—This paper explains the design principles and implementation strategies behind a complete navigation system for autonomous maze-solving robots. Rather than focusing on mathematical proofs, we describe the practical reasoning behind each architectural decision: why the grid is represented as it is, how the dual-mode wall beliefs enable both safe navigation and effective exploration, why the flood-fill heuristic makes A* so efficient, how Trémaux exploration guarantees complete coverage, why clothoid curves are necessary for smooth physical motion, and how a Kalman filter fuses unreliable sensors into a reliable state estimate. The goal is to provide an accessible explanation of the system's working principles for engineers implementing similar navigation stacks.**

***Index Terms*—A* search, admissible heuristics, clothoid curves, Dijkstra's algorithm, Kalman filtering, maze solving, path planning, Trémaux algorithm**

I. Introduction

When building an autonomous maze-solving robot, the fundamental challenge is not any single algorithm—it's the integration of many algorithms into a coherent system. The robot must explore an unknown environment, build a map, plan optimal paths, and execute smooth physical trajectories, all while maintaining an accurate estimate of its own position. Each of these tasks requires different algorithmic approaches, and the interfaces between them must be carefully designed.

This paper explains the working principles of a complete navigation stack, focusing on the practical reasoning behind each design choice. We describe not just what each component does, but why it was chosen over alternatives and how it connects to the other components. The architecture consists of six main subsystems:

1) **Grid World Model:** How the robot represents its environment internally, including the crucial distinction between what it knows and what it assumes.
2) **Flood-Fill Distance Field:** How the robot precomputes a "distance to goal" map that guides all its navigation decisions.
3) **A* Path Planning:** How the robot finds the optimal path through its known map, using the precomputed distances to search efficiently.
4) **Trémaux Explorer:** How the robot systematically explores unknown territory with a guaranteed-complete strategy.
5) **Clothoid Trajectories:** How the robot converts a sequence of waypoints into a smooth, physically-drivable path.
6) **Kalman Filter:** How the robot combines noisy wheel encoders with wall-distance measurements to know where it actually is.

II. The Grid World Model: How the Robot Sees Its Environment

A. Why a Grid?

The first design decision is how to represent the maze. The continuous physical world is discretized into a 16×16 grid of cells, each 0.18 m per side. This is a classic tradeoff: a grid simplifies planning to a graph search problem, while being fine enough to capture all relevant maze geometry. The grid cell becomes the atomic unit of position—the robot is always considered to be in exactly one cell.

B. Linear Indexing for Performance

Although cells are logically addressed by (x, y) coordinates, they are stored in a 1D array using the mapping $\text{idx}(x, y) = y \times 16 + x$. This is a performance optimization: 1D array access is faster and more cache-friendly than 2D array access, and many algorithms in the system iterate over all cells repeatedly. The conversion is $O(1)$ and happens transparently.

C. The Dual-Mode Wall Belief System

This is perhaps the most important design concept in the entire system. Each cell has four walls (North, East, South, West), and for each wall, the robot maintains two boolean values:

- **The actual state** W: Is there physically a wall here? (0 = open passage, 1 = wall)
- **The knowledge state** K: Has the robot observed this wall? (0 = unknown, 1 = confirmed)

The power of this separation is that it enables two completely different world views from the same data:

The Pessimistic View (for safe, high-speed navigation): If the robot hasn't seen a wall, it assumes the wall exists. In code terms, a passage is only traversable if $K = 1$ AND $W = 0$—the robot must have confirmed the wall is open. This means the robot will never accidentally drive into an unseen

wall during the speed run. The pessimistic view is used when the robot's priority is safety over exploration.

The Optimistic View (for exploration): If the robot hasn't seen a wall, it assumes the passage is open. In code terms, a passage is traversable if $K = 0$ OR $W = 0$—either it's unknown (and therefore worth exploring) or known to be open. This is what drives the robot to venture into uncharted territory during the exploration phase.

D. Wall Symmetry: One Observation, Two Updates

A physical wall between two cells is a shared boundary. When the robot observes a wall to its North, that same wall is simultaneously the South wall of the neighboring cell. The system enforces this symmetry automatically: setting $W(x, y, \mathrm{N}) = 1$ also sets $W(x, y-1, \mathrm{S}) = 1$, and both cells get $K = 1$ for those walls. This doubles the mapping efficiency—every observation updates two cells.

E. Diagonal Movement and Corner Clearance

The robot can move diagonally (NE, SE, SW, NW), but it must not "clip" through corners. To move Northeast, both the North wall AND the East wall of the current cell must be open. This prevents the robot from squeezing through a gap that only exists because of the grid discretization.

III. The Flood-Fill: Precomputing Distance to Goal

A. The Core Idea

Before the robot can navigate efficiently, it needs a sense of direction. The flood-fill algorithm computes, for every cell in the maze, the exact shortest distance to the goal. This creates a "potential field" where each cell's value represents how far it is from the goal. Lower numbers are better—they mean you're closer.

B. How It Works

The algorithm starts by marking all goal cells with distance 0. It then uses Dijkstra's algorithm to expand outward, cell by cell:

1) Take the cell with the smallest known distance from a priority queue.
2) For each of its 8 neighbors that is reachable (using the optimistic assumption):
   - Calculate the distance through this cell: current distance + move cost (1 for cardinal, $\sqrt{2}$ for diagonal).
   - If this is shorter than the neighbor's current distance, update it and add it to the queue.
3) Repeat until all reachable cells have been processed.

The result is a map where every cell "knows" its distance to the nearest goal. This computation happens in milliseconds for a 16×16 grid and is re-run whenever the map changes significantly.

C. Why This Makes A* So Fast

The flood-fill values serve as the heuristic function for A* pathfinding. A heuristic estimates the remaining distance from any cell to the goal. Because the flood-fill was computed on the fully-open graph (optimistic assumption), it never overestimates the true distance on the real, walled map. This property—admissibility—guarantees that A* will find the optimal path. Moreover, because the heuristic is very accurate (it's the exact distance on a slightly more open version of the maze), A* expands very few cells—it essentially heads straight for the goal.

D. The Return-to-Start Variant

The same algorithm can compute distances from the start cell instead of to the goal. By seeding the start cell with distance 0 and running Dijkstra, every cell gets its distance from start. This is used by the Trémaux explorer when it needs to find its way back to known territory.

IV. A* Path Planning: Optimal Navigation on a Known Map

A. How A* Uses the Flood-Fill

A* is a best-first search algorithm. It maintains a priority queue of cells to explore, sorted by $F(c) = g(c) + h(c)$, where:

- $g(c)$ is the actual distance traveled from the start to cell c (following the actual path).
- $h(c)$ is the heuristic estimate—in our case, the flood-fill distance from c to the goal.

The algorithm repeatedly takes the most promising cell (lowest F), explores its neighbors, and updates their scores. The flood-fill heuristic $h(c)$ guides the search toward the goal, while $g(c)$ keeps it honest about the actual path cost. Because the heuristic is both admissible and consistent, A* is guaranteed to find the optimal path.

B. Handling Stale Queue Entries Efficiently

A subtle implementation detail: a cell might be discovered via multiple paths, each with a different F-score. Rather than updating existing entries in the priority queue (which is expensive), the implementation simply pushes a new, better entry and leaves the old one. When an entry is popped, the code checks whether its stored g-cost matches the best known g-cost for that cell. If not, the entry is stale and is discarded. This "lazy deletion" strategy is much faster than decrease-key operations.

C. Path Reconstruction

As A* expands cells, it records which cell led to each neighbor (the parent). When the goal is reached, the path is reconstructed by following parent pointers backward from the goal to the start, then reversing the sequence.

V. Path Post-Processing: From Cells to Smooth Paths

A. Run-Length Encoding

The raw A* output is a sequence of 256 or fewer cells. Consecutive steps in the same direction are compressed into a single "move" command with a distance. For example, "East, East, East, Northeast, North, North" becomes two moves: "East $\times$ 3" and "Northeast $\times$ 1, North $\times$ 2." This compression is necessary for the geometric smoothing that follows.

B. String-Pulling Smoothing

The compressed path may have unnecessary bends. The string-pulling algorithm works like tightening a string through the maze corridors: it removes intermediate waypoints whenever a direct line-of-sight exists from the last kept point to a future point. The line-of-sight check is conservative—it only considers straight lines (axis-aligned or 45-degree diagonals) where every intermediate cell is known to be passable. The algorithm repeats until no more points can be removed, resulting in a path with the minimum number of turns.

VI. Trémaux Exploration: Guaranteed Complete Mapping

A. The Exploration Problem

Before the robot can plan optimal paths, it must discover the maze layout. The challenge is to explore systematically without missing any passages and without getting stuck in infinite loops. The Trémaux algorithm, dating from the 19th century, solves this elegantly.

B. The Core Rule: Prefer the Unknown

The fundamental principle is simple: when choosing where to go next, always prefer a cell you haven't visited before. This single rule guarantees that the robot will eventually visit every reachable cell. The implementation uses a visit counter for each cell; the primary sorting criterion is the visit count, with unvisited cells (count = 0) getting highest priority.

C. Adding Intelligence with Scoring

Pure Trémaux with random tie-breaking would work, but it would be inefficient. Our implementation adds two secondary scoring criteria:

1) **Distance to goal** (from the flood-fill): Among equally-unvisited cells, prefer the one closer to the goal. This biases exploration toward the goal region.
2) **Information gain**: Prefer cells that will reveal more unknown walls. A cell at a junction into unexplored territory has high information gain; a cell in a narrow, already-mapped corridor has low gain. This metric counts how many unknown walls would become visible from the cell.

These criteria form a lexicographic sort: visits first, then distance, then information gain. The explorer is still guaranteed to be complete (the visit count ensures that), but it explores much more intelligently than random selection.

D. Backtracking: What to Do When Stuck

When all reachable neighbors have been visited, the robot is at a dead end of exploration. It must backtrack to the last junction that still has unexplored branches. The implementation maintains a stack of visited cells. When stuck, it pops cells from the stack until finding one with an unvisited neighbor, then uses A* on the pessimistic (known-safe) map to navigate there efficiently. This is faster than blindly retracing the original exploration path.

E. Why It Terminates

The Trémaux property guarantees that each edge in the maze graph is traversed at most twice (once going in, once coming out). Since the maze has a finite number of edges (at most $4 \times 256 = 1024$ cardinal connections), the exploration will always terminate with every reachable cell visited.

VII. Clothoid Trajectories: From Waypoints to Smooth Motion

A. Why Simple Waypoints Fail

A path defined as a sequence of waypoints with instantaneous turns is physically impossible to execute at speed. If the robot is moving at 1 m/s and is told to turn 90 degrees at a waypoint, it would require infinite acceleration. Real robots need smooth transitions.

B. What Makes Clothoids Special

A clothoid (or Cornu spiral) is a curve whose curvature increases linearly with distance traveled. Curvature κ is the inverse of turn radius: straight line = $\kappa = 0$, tight turn = large κ. In a clothoid, κ changes at a constant rate, which means the steering angle changes at a constant rate. This produces a natural, smooth transition from straight to turning and back.

The key property is C^2 continuity: both the path's direction (first derivative) and curvature (second derivative) are continuous. There are no sudden jumps in lateral force—the robot transitions smoothly into and out of turns.

C. Building a Turn from Three Pieces

A complete turn between two straight segments is constructed from three parts:

1) **Entry Clothoid:** Curvature ramps linearly from 0 (straight) to the desired turn curvature. The robot smoothly begins turning.
2) **Circular Arc:** Curvature stays constant. This is the main body of the turn, where the robot sweeps through most of the heading change.
3) **Exit Clothoid:** Curvature ramps linearly back to 0. The robot smoothly straightens out.

The lengths of the clothoid segments are computed from the turn's geometry. A rule of thumb uses $L_c = \sqrt{|\kappa_{\text{turn}}|/5}$, capped at 40% of the total segment length to leave room for the circular arc. The three pieces are stitched together so that curvature is continuous at every junction.

D. Numerical Integration for Position

The clothoid's position cannot be expressed in closed form—it involves Fresnel integrals. The implementation approximates the position by numerically integrating the heading angle using 10 rectangular samples per segment. This is computationally cheap and accurate enough for the 0.18 m grid scale.

VIII. Jerk-Limited Velocity Profiling

A. The Speed Limit Problem

Even with a smooth path, the robot cannot travel at constant speed. Sharp turns require slower speeds because of centripetal force: $a_{\text{lateral}} = \kappa v^2$. If the curvature κ is high, the speed v must be low to keep the lateral acceleration within the tires' grip limit (8.0 m/s^2).

B. The Two-Pass Solution

The velocity profiler solves this with a forward pass followed by a backward pass:

Forward Pass: Starting from $v = 0$ at the path start, the algorithm propagates forward, computing the maximum possible speed at each point based on three constraints:

- The centripetal limit: $v \leq \sqrt{a_{\text{lat}}/|\kappa|}$
- The acceleration limit: the robot cannot speed up faster than 9.0 m/s^2 or with jerk exceeding 80 m/s^3
- The top speed: 5.0 m/s

Backward Pass: Starting from $v = 0$ at the path end (the robot must stop at the goal), the algorithm propagates backward, reducing speeds where necessary to ensure the robot can brake in time. The braking deceleration is limited to 10.0 m/s^2 with the same jerk limit.

The final speed at each point is the minimum of the forward-pass speed and the backward-pass speed. This guarantees the profile is both physically achievable and safe—the robot can always stop by the end.

C. Why Two Passes Are Necessary

The forward pass alone cannot guarantee safety because it doesn't know about upcoming sharp turns or the final stop. Consider a long straightaway followed by a hairpin turn: the forward pass would accelerate to top speed on the straight, not knowing it needs to slow down for the turn. The backward pass fixes this by propagating the braking requirement backward from the turn. The two passes together produce a globally valid speed profile.

IX. Kalman Filter: Knowing Where You Are

A. The Sensor Fusion Problem

The robot has two sources of position information, neither of which is reliable alone:

- **Wheel encoders:** Measure how far each wheel has turned, giving distance Δs and heading change $\Delta\theta$. These are accurate over short distances but drift over time—small errors accumulate.
- **Wall distance sensors:** Measure the distance to nearby walls. These provide absolute position references (the walls don't move), but only when the robot is driving alongside a known wall.

The Kalman filter combines these optimally: it trusts the encoders for short-term motion and uses wall measurements to correct accumulated drift.

B. The Three-State Model

The filter estimates three values: x position, y position, and heading θ. Each has an associated uncertainty (variance). The filter maintains a 3×3 covariance matrix $\mathbf{P}$, though in practice it uses a diagonal approximation since the cross-correlations between x, y, and θ errors are small.

C. The Predict-Update Cycle

Predict Step (happens continuously): Whenever the robot moves, the encoders provide Δs and $\Delta\theta$. The filter predicts the new position by integrating this motion, using a midpoint heading model for better accuracy. Crucially, it also increases the uncertainty—moving makes the robot less certain about its exact position, and the uncertainty grows proportionally to the distance traveled.

Update Step (happens when near a known wall): When the robot drives alongside a wall it has mapped, its side sensors measure the distance to that wall. The filter compares this measurement with the distance it expected based on its current position estimate. The difference (the innovation) is used to correct the position estimate. The Kalman gain determines how much to trust the measurement versus the prediction: if the prediction is very uncertain (large covariance), the measurement gets more weight; if the prediction is confident, the measurement gets less weight.

D. Why This Works

The Kalman filter is optimal in the sense that it minimizes the mean squared estimation error, assuming Gaussian noise. The encoder noise is modeled as proportional to motion (longer moves = more uncertainty), and the wall sensor noise is a constant 0.003 m^2. The filter continuously balances these two information sources, maintaining the best possible position estimate at all times.

X. System Integration: How It All Fits Together

A. The Exploration Phase

When the robot starts, its map is entirely unknown. The system operates as follows:

1) The Trémaux explorer selects the next cell to visit based on the scoring function, using the optimistic passability (unknown = open).
2) As the robot moves, it observes walls and updates the map using the symmetry constraint.
3) The flood-fill is recomputed periodically to reflect the growing map, keeping the heuristic accurate.
4) When stuck, the explorer uses A* on the pessimistic map to backtrack to the nearest unexplored junction.
5) This continues until all reachable cells are visited.

B. The Speed Phase

With the map complete, the robot switches to optimal navigation:

1) A* finds the shortest path from start to goal on the pessimistic (fully-known) map.
2) The path is run-length encoded and smoothed with string-pulling.
3) Clothoid curves are generated to smooth all turns.
4) The velocity profiler computes jerk-limited speeds along the trajectory.
5) The Kalman filter provides continuous position estimates during execution.

C. Computational Requirements

All algorithms are designed to run on embedded hardware. Table I shows the complexity of each component. The flood-fill and A* are the most expensive operations at $O(N^2 \log N)$, but for $N = 16$ this is negligible (approximately 2500 operations). The Kalman filter runs in constant time per update. The entire system can execute in real-time on a microcontroller.

TABLE I
COMPUTATIONAL COMPLEXITY SUMMARY

Component	Time	Guarantee
Flood-fill	$O(N^2 \log N)$	Exact distances
A* Planner	$O(N^2 \log N)$	Optimal path
Path Smoother	$O(n^2)$	Minimum turns
Trémaux	$O(N^2)$	Full coverage
Clothoid Gen.	$O(T)$	C^2 continuity
Velocity Prof.	$O(T)$	Jerk-limited
Kalman Filter	$O(1)$/step	Optimal estimate

XI. CONCLUSION

This paper has explained the design principles behind a complete maze-solving navigation system. The key architectural insights are:

- **Dual-mode wall beliefs** enable a single map to serve both safe navigation (pessimistic) and exploration (optimistic).
- **Precomputed distance fields** provide an accurate, admissible heuristic that makes A* extremely efficient.
- **Trémaux exploration with heuristic scoring** guarantees complete coverage while exploring intelligently.
- **Clothoid curves** provide physically-smooth trajectories that respect the robot's dynamics.
- **Two-pass velocity profiling** ensures speeds are both achievable and safe.
- **Kalman filtering** optimally fuses noisy encoder data with absolute wall measurements.

The system is designed as a set of independent components with well-defined interfaces: the map feeds the flood-fill, the flood-fill feeds A* and the explorer, the path feeds the trajectory generator, and the trajectory executes with continuous Kalman position estimates. This modularity allows each component to be developed, tested, and verified independently while guaranteeing correct behavior when integrated.

REFERENCES

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[4] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *J. Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[5] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[6] A. Kelly and B. Nagy, "Reactive Nonholonomic Trajectory Generation via Parametric Optimal Control," *Int. J. Robotics Research*, vol. 22, no. 7, pp. 583–601, 2003.